

ML Auto Flow

ML Auto Flow Project

June 23, 2026

CONTENTS

ML Auto Flow	
브라우저에서 즉시 실행되는 시각적 머신러닝 파이프라인	
차례	
머리말 — 데이터 과학의 민주화, 그리고 클라우드 없는 길	
제1부 · 데이터 과학의 기초	
1장 — 데이터의 과학: 머신러닝이란 무엇인가	
프로그램을 “짜는” 대신 “기르는” 일	
우리 일상에 스며든 예측 분석	
학습의 형태 — 지도·비지도로 넘어	
“실패를 빨리(fail fast), 그리고 계속 배우는” 기계	
2장 — 머신러닝은 어떻게 작동하는가	
반복 가능한 워크플로	
알고리즘의 세 갈래	
지도학습: 정답을 보고 배우기	
평가: “완벽”이 아니라 “충분히 가까움”	
일반화 — 외운 것이 아니라 배운 것	
비지도학습: 혼돈 속에서 숲을 찾기	
제2부 · ML Auto Flow 시작하기	

3장 — 캔버스 둘러보기	
가입 절차 대신, 그냥 열기	
화면의 구성요소	
모듈과 포트, 그리고 흐름	
4장 — 데이터 들여오기와 탐색	
LoadData — 파일 또는 URL	
데이터 개요 패널 — 먼저 데이터를 “본다”	
결측치 다루기	
데이터의 종류와 전처리의 논리	
제3부 · 지도학습: 예측 모델 만들기	
5장 — 분류: 성인 소득(>50K) 예측	
시나리오	
파이프라인	
의사결정나무는 어떻게 나누나	
앙상블 — 약한 학습기를 모아 강하게	
검증된 결과 (test n = 6,513)	
분류 평가 깊이 보기 — 정확도만으로는 부족하다	
그래디언트 부스팅 — 책의 1순위 분류기에 대응	
내보낸 동등 Python (검증됨, 핵심 발췌)	
6장 — 회귀: 자동차 가격 예측	

시나리오

회귀의 세 갈래(개념)

파이프라인

검증된 결과 (test n = 51)

회귀 평가 깊이 보기 — 상대오차의 추가

선형회귀가 기대는 가정, 그리고 규제

내보낸 동등 Python (검증됨)

회귀 알고리즘 패밀리(개념 지도)

제4부 · 비지도학습

7장 — 군집: 도매고객 세분화

정답이 없을 때

K-Means의 원리

시나리오와 파이프라인

검증된 결과

군집 패밀리 — K-Means를 넘어

제5부 · 책 너머로: 통계와 진단

8장 — 통계 분석과 가설검정

제6부 · ML Auto Flow의 차별화된 강점

9장 — Python 코드 내보내기과 재현성 ★

재현성 불변식

자동 회귀 검증 — 약속을 기계가 지킨다	
하이퍼파라미터 스윙 — 자동 튜닝도 결정적으로	
10장 — AI 보조 기능: 코드·결과·오류를 한국어로 ★	
11장 — 모델을 서비스로: 스코어링 코드 내보내기	
12장 — 추천 시스템: 협업 필터링과 행렬분해	
추천 엔진은 왜 강력한가	
두 가지 방법론	
희소 행렬과 콜드 스타트	
행렬분해 — 잠재요인으로 빈칸 메우기	
ML Auto Flow의 Recommender — NMF 행렬분해	
예제: 레스토랑 평점 추천 — 처음부터 끝까지	
13장 — 모델 재학습과 지속학습	
피드백 루프가 모델을 살린다	
ML Auto Flow의 재학습 — 버전이 매겨진 모델 스냅샷	
부록	
부록 A — 모듈 레퍼런스	
부록 B — 예제 데이터셋과 검증 픽스처	
부록 C — Azure ML Studio ↔ ML Auto Flow 용어 대조표	
부록 D — 변경 이력	
제작 메모 (판단 사항)	

ML Auto Flow

브라우저에서 즉시 실행되는 시각적 머신러닝 파이프라인

— 클라우드 가입도, 설치도, 과금도 없는 예측 분석 입문 —

이 책에 대하여

이 책자는 Jeff Barnes의 *Microsoft Azure Essentials: Azure Machine Learning*(Microsoft Press, 2015)이 제시한 **학습 흐름**(데이터 → 시각화 → 분할 → 학습 → 채점 → 평가 → 배포 → 재 학습)을 길잡이 삼아, 같은 여정을 **ML Auto Flow** 앱으로 처음부터 끝까지 걸어 보는 실습 안내서다.

Azure ML Studio가 *클라우드 위의 드래그-앤-드롭 캔버스*라면, ML Auto Flow는 ****여러분의 브라우저 안에서 도는 캔버스 + 브라우저 내 Python(Pyodide)****이다. 가입·결제·서버 프로비저닝 없이, 웹페이지 하나만 열면 같은 개념을 곧바로 손에 익힐 수 있다.

본문의 모든 **수치·코드·결과**는 앱이 실제로 내보내는 Python을 외부에서 2회 실행해 ****바이트 단위로 동일(byte-identical)****한지 자동 검증한(`npm run verify:pipelines`, **15/15 PASS**) 것만 실었다. “책에 적힌 대로 실행하면 같은 결과”가 이 책의 약속이다.

차례

머리말 — 데이터 과학의 민주화, 그리고 클라우드 없는 길

제1부 · 데이터 과학의 기초

- 1장 데이터의 과학 — 머신러닝이란 무엇인가
- 2장 머신러닝은 어떻게 작동하는가 — 워크플로와 두 갈래 학습

제2부 · ML Auto Flow 시작하기

- 3장 캔버스 둘러보기 — 모듈·포트·실행·미리보기
- 4장 데이터 들어오기와 탐색 — LoadData와 데이터 개요

제3부 · 지도학습: 예측 모델 만들기

- 5장 분류 — 성인 소득(>50K) 예측 (end-to-end)
- 6장 회귀 — 자동차 가격 예측 (end-to-end)

제4부 · 비지도학습

- 7장 군집 — 도매고객 세분화

제5부 · 책 너머로: 통계와 진단

- 8장 통계 분석과 가설검정

제6부 · ML Auto Flow의 차별화된 강점

- 9장 Python 코드 내보내기와 재현성 ★
- 10장 AI 보조 기능 — 코드·결과·오류를 한국어로 ★
- 11장 모델을 서비스로 — 스코어링 코드 내보내기
- 12장 추천 시스템 — 협업 필터링과 행렬분해
- 13장 모델 재학습과 지속학습

부록

- 부록 A 모듈 레퍼런스
- 부록 B 예제 데이터셋과 검증 픽스처
- 부록 C Azure ML Studio ↔ ML Auto Flow 용어 대조표
- 부록 D 변경 이력

머리말 — 데이터 과학의 민주화, 그리고 클라우드 없는 길

10여 년 전만 해도 예측 모델 하나를 만들어 운영에 올리는 일은 전담 데이터 과학자 팀, 전용 인프라, 그리고 수 주에서 수 개월의 시간을 요구했다. *Azure Machine Learning*이 그 풍경을 바꾼 핵심은 **데이터 과학의 민주화**였다. 누구나 브라우저에서 모듈을 끌어다 놓고 선으로 잇기만 하면 정교한 예측 분석 실험을 구성하고, 클릭 몇 번으로 웹 서비스로 내보낼 수 있게 한 것이다.

ML Auto Flow는 그 정신을 한 걸음 더 끌고 간다. **클라우드 계정도, 신용카드도, 서버도 필요 없다**. 모든 계산은 여러분의 브라우저 안에서 도는 Python(Pyodide·WebAssembly)으로 수행되고, 데이터는 여러분의 기기를 떠나지 않는다. 학습용 노트북 한 대, 혹은 사내 망 안에서도 “데이터를 올리고 → 모델을 만들고 → 평가하고 → 코드로 내보내는” 전 과정을 그대로 체험할 수 있다.

이 책이 책과 다른 네 가지

1. **클라우드·과금 불필요** — 가입·결제·프로비저닝 절차가 통째로 “브라우저 열기” 한 줄로 대체된다.
2. **브라우저 내 Python 실행(Pyodide)** — `numpy` · `pandas` · `scikit-learn` · `statsmodels`가 설치 없이 동작한다.
3. **재현성 보증** — “전체 코드 보기”로 내보낸 Python이 외부에서 **동일 결과**를 내며, 자동 회귀 검증으로 늘 지켜진다.
4. **AI 한국어 해설** — 코드·결과·오류를 그 자리에서 설명한다(사용자별 로컬 키로 동작, 고급기능 게이트로 보호).

이 책은 무거운 수식 증명에 매달리지 않는다. 대신 **개념의 직관과 그 근거**, 그리고 **손으로 익히는 실습**을 함께 담는다. 각 장은 **왜 그렇게 동작하는가(이론)** → 앱에서의 실습 → 내보낸 코드 → 결과 해석의 네 박자로 진행되며, 곳곳의 사이드바(> 박스)에 한 걸음 더 들어간 이론을 따로 묶었다. 앞 장을 건너뛰어도 특정 장만 따로 읽을 수 있도록 구성했다.

제1부 · 데이터 과학의 기초

1장 — 데이터의 과학: 머신러닝이란 무엇인가

프로그램을 “짜는” 대신 “기르는” 일

전통적인 프로그래밍에서는 **사람이 규칙(프로그램)을 쓰고**, 컴퓨터가 그 규칙에 데이터를 넣어 결과를 만든다. 규칙은 명시적이고, 사람이 모든 경우를 미리 생각해 코드로 적어야 한다.

머신러닝은 이 방향을 뒤집는다. **데이터와 “정답(원하는 결과)”을 함께 주면**, 컴퓨터가 그 둘을 잇는 규칙(모델)을 스스로 찾아낸다. 일단 찾아낸 규칙은 처음 보는 입력에 대해서도 결과를 **예측**할 수 있다. 한마디로 머신러닝은 **데이터를 소프트웨어로 바꾸는 방법**이며, 그 바탕 기술이 **예측 분석(predictive analytics)** — 과거를 과학적으로 활용해 미래를 가늠하는 일이다.

	전통적 프로그래밍	머신러닝
사람이 주는 것	규칙 + 데이터	데이터 + 정답(레이블)
컴퓨터가 만드는 것	결과	규칙(모델)
새 입력에 대해	정해진 대로 처리	예측

규칙이 너무 많거나 모호해서 사람이 일일이 적기 어려운 문제 — 손글씨 주소 판독, 스팸 분류, 신용 위험 평가 — 일수록 머신러닝의 값어치가 커진다.

우리 일상에 스며든 예측 분석

예측 분석은 이미 우리 주변 곳곳에 있다. 몇 가지만 꼽아도:

- **스팸/정크 메일 필터** — 내용·헤더·발신 패턴·사용자 행동으로 분류.
- **신용·대출 심사** — 상환 이력과 인구통계로 위험을 점수화.
- **문자/음성/얼굴 인식** — 우편물 자동 분류(OCR), 스마트폰 음성 비서, 보안 시스템.
- **보험** — 생명보험의 사망률·기대수명·보험료 산정, 의료비 예측, 손해보험 위험 평가.
- **부정거래 탐지** — 사용·활동 패턴으로 이상 거래 식별.
- **추천** — 전자상거래의 “이 상품을 산 사람은...”, 영상 스트리밍의 개인화 홈.
- **예지정비(predictive maintenance)** — 항공기·엘리베이터·데이터센터의 고장 예측.

특히 **보험·계리** 영역은 회귀로 위험을 모델링해 온 오랜 역사를 갖고 있다. 예컨대 피보험자당 청구 건수를 연령 같은 요인에 회귀하면, 고연령 고객의 청구가 늘어나는 경향을 수치로 드러낼 수 있다. ML Auto Flow의 회귀·통계 모듈은 바로 이런 분석을 손으로 재현하게 해 준다.

학습의 형태 — 지도·비지도를 넘어

머신러닝을 조금 더 형식적으로 보면, **입력 X 에서 출력 Y 로 가는 함수 f 를 데이터로 근사하는 일**이다. 어떤 신호를 주느냐에 따라 학습은 크게 네 갈래로 나뉜다.

- **지도학습(supervised)** — (X, Y) 쌍이 주어진다. 정답 Y 를 보고 f 를 맞춰 간다(이 책 5·6장).
- **비지도학습(unsupervised)** — X 만 주어진다. 데이터 안의 구조(군집·저차원 표현)를 찾는다(7장).
- **준지도학습(semi-supervised)** — 소수의 레이블 + 다수의 비레이블. 레이블링 비용이 클 때 쓴다.
- **강화학습(reinforcement)** — 정답 대신 **보상**이 주어진다. 시행착오로 정책을 학습한다(게임·로봇 제어).

ML Auto Flow는 실무에서 가장 자주 쓰는 앞의 두 갈래에 더해 **추천(12장)·통계(8장)** 까지 캔버스로 다룬다.

왜 데이터가 많을수록 좋은가 — 그리고 그 한계 모델은 표본에서 모집단의 규칙을 추정한다. 표본이 크고 다양할수록 추정의 분산이 줄어 (큰 수의 법칙) 처음 보는 데이터에 대한 예측이 안정된다. 다만 **양보다 질** — 편향된 데이터는 아무리 많아도 편향된 모델을 낳는다. 5장의 클래스 불균형(고소득자가 소수)이 그 산 증거다.

”실패를 빨리(FAIL FAST), 그리고 계속 배우는” 기계

머신러닝의 가장 독특한 점은 **끝이 없다**는 데 있다. 모델은 새 데이터가 들어올 때마다 오차를 피드백받아 자신을 고쳐 나간다. 사람과 달리, 기계는 같은 실수를 반복하도록 운명 지어지지 않는다. 그래서 좋은 예측 시스템의 핵심은 **빠른 피드백 루프**다. 가설을 세우고, 빨리 실험하고, 틀렸으면 빨리 버리고(fail fast), 맞으면 다듬는다.

ML Auto Flow의 캔버스는 이 “빠른 실험” 사이클을 손쉽게 만든다. 모듈을 바꿔 끼우고 ▶ 실행만 누르면 새로운 가설을 곧바로 시험할 수 있다. 클라우드에 배포할 필요도, 결과를 기다릴 필요도 없다 — 계산은 브라우저 안에서 즉시 끝난다.

한눈에 머신러닝 = 데이터에서 규칙을 학습해 예측·분류·군집을 수행하는 기술. 핵심 동력은 ① 폭증하는 데이터 ② 값싼 저장소 ③ 어디서나 쓰는 연산력 ④ 빅데이터 분석의 경제성. ML Auto Flow는 ②③을 **여러분의 브라우저**로 대체해 진입장벽을 0에 가깝게 낮춘다.

2장 — 머신러닝은 어떻게 작동하는가

반복 가능한 워크플로

성공적인 예측 분석은 즉흥이 아니라 **반복 가능한 절차**를 따른다. 그 절차는 대략 이렇게 흐른다.

1. **데이터(Data)** — 학습·검증용 데이터를 모으고, 살펴보고, 정리한다. 모든 것은 데이터에서 시작한다.
2. **모델 생성(Create)** — 알고리즘으로 데이터의 패턴을 학습해 예측 모델을 만든다.
3. **평가(Evaluate)** — 입력과 정답이 모두 알려진 데이터로 정확도를 잰다. 정확도는 0~1 사이 신뢰도로 표현된다.
4. **정제·비교(Refine)** — 여러 모델·전략을 비교·결합해 가장 일관되게 정확한 조합을 찾는다.
5. **배포(Deploy)** — 모델을 어디서나 호출할 수 있는 형태(웹 서비스/스코어링 코드)로 내보낸다.
6. **테스트·활용(Use)** — 실제 시나리오에 적용하고, 새 결과를 피드백으로 되먹여 모델을 계속 개선한다.

ML Auto Flow는 이 6단계를 **캔버스 위 모듈**로 1:1 대응시킨다. 데이터 적재(`LoadData`) → 전처리·분할(`SplitData`) → 모델 정의 → 학습(`TrainModel`) → 채점(`ScoreModel`) → 평가(`EvaluateModel`) → 코드/서비스 내보내기. 6단계를 눈으로 보면서 손으로 잇는다.

알고리즘의 세 갈래

ML Auto Flow가 제공하는 학습 알고리즘은 크게 셋으로 나뉜다.

- **분류(Classification)** — 출력이 **몇 개의 범주 중 하나**일 때. 예: 소득 `>50K` vs `<=50K`, 이탈/잔존.
- **회귀(Regression)** — 출력이 **연속적인 수치**일 때. 예: 자동차 가격, 손익, 청구액.
- **군집(Clustering)** — 정답 없이 데이터 안의 **자연스러운 묶음**을 찾을 때. 예: 고객 세분화.

앞의 둘(분류·회귀)은 **지도학습**, 마지막(군집)은 **비지도학습**에 속한다.

지도학습: 정답을 보고 배우기

지도학습은 **입력과 정답이 함께 있는** 데이터(학습 데이터)로 모델을 훈련한다. 학습 데이터의 각 열(column)은 세 가지 역할 중 하나를 맡는다.

- **특성(Features) / 입력 벡터** — 예측에 쓰이는 입력 변수들.
- **레이블(Label) / 정답 신호** — 그 입력에 대응하는 알려진 결과.
- **미사용(Not used)** — 이번 예측에는 쓰지 않는 열(나중에 필요하면 다시 포함 가능).

예를 들어 **성인 인구조사 소득** 데이터(5장)에서는 나이·학력·직업·결혼상태·주당 근로시간 같은 열이 특성이 되고, 소득이 `>50K` 인지 `<=50K` 인지가 **레이블**이 된다. 모델은 “어떤 특성 조합이 어떤 결과로 이어지는가”라는 **반복 가능한 추론 패턴**을 찾아낸다.

ML Auto Flow에서 특성과 레이블의 지정은 `TrainModel` 모듈의 `feature_columns` 와 대상 열(target) 설정으로 이뤄진다. 책의 *Project Columns*(열 선택)에 해당하는 역할이다.

모델은 어떻게 “맞춰지나” — 손실과 최적화 학습이란 결국 *예측이 정답에서 벗어난 정도*(손실, *loss*)를 최소화하도록 모델 내부의 수(파라미터)를 조정하는 일이다. 선형회귀는 제공오차 손실을 **단한 해(최소제곱)**로 한 번에 풀고, 신경망·부스팅은 **경사하강(gradient descent)**으로 손실의 기울기를 따라 조금씩 내려간다. 초기화·표본추출에 무작위가 끼는 단계마다 시드를 고정 (`random_state=42`)하는 이유가 여기에 있다(9장). 같은 출발점에서 같은 길을 내려가야 같은 답에 도달하기 때문이다.

평가: “완벽”이 아니라 “충분히 가까움”

새 모델을 만들면 **가장 먼저** 정확도를 평가해야 한다. 입력과 정답이 모두 알려진 검증 데이터로 모델의 예측이 얼마나 들어맞는지 재는 것이다. 중요한 진실은 — **모델은 결코 100% 완벽하지 않다**. 오히려 학습 데이터에서 100%가 나오면 과적합(overfitting)을 의심해야 한다. 현실의 핵심은 *처음 보는, 예측이 있는 데이터*에 대해 얼마나 잘 맞히느냐다. 그래서 우리는 “현실적으로 받아들일 수 있는 정확도 범위”를 정하고 거기에 맞춰 모델을 다듬는다.

분류에서는 정확도·정밀도·재현율·F1·혼동행렬·ROC/AUC를, 회귀에서는 R^2 ·RMSE·MAE·RSE·RAE를 본다(각각 5·6장에서 실제 수치와 함께 다룬다). ML Auto Flow의 `EvaluateModel`이 이 지표들을 한번에 계산한다.

일반화 — 외운 것이 아니라 배운 것

평가가 던지는 진짜 질문은 “모델이 데이터를 *외웠는가*, 아니면 규칙을 *배웠는가*”이다. 학습 데이터에만 잘 맞고 새 데이터에 약한 상태를 **과적합(overfitting)**, 학습 데이터조차 제대로 못 맞히는 상태를 **과소적합(underfitting)**이라 한다.

- **과적합** — 모델이 너무 복잡해 잡음까지 외운다(예: 깊은 의사결정나무). 학습 정확도↑, 검증 정확도↓.
- **과소적합** — 모델이 너무 단순해 패턴을 놓친다. 학습·검증 모두 낮음.

이 둘 사이의 줄다리기가 **편향-분산 트레이드오프(bias-variance tradeoff)**다. 단순한 모델은 *편향*이 크고(체계적으로 빗나감), 복잡한 모델은 *분산*이 크다(데이터가 조금만 바뀌어도 결과가 출렁인다). 좋은 모델은 그 사이 적절한 지점에 있다 — 그리고 그 지점은 **데이터로 찾는 것**이지 손으로 정하는 게 아니다.

그래서 데이터를 **학습/검증/테스트**로 나눈다(이 책은 학습/테스트 2분할을 쓴다).

- **학습(train)**으로 모델을 적합하고,
- **테스트(test)**로 *처음 보는* 데이터에 대한 성능을 잰다.
- 모델·하이퍼파라미터를 여러 번 비교할 때는 **교차검증(cross-validation)** — 데이터를 K조각으로 나눠 번갈아 가며 검증 — 으로 더 안정적으로 고른다(9장의 `SweepParameters`가 `GridSearchCV`로 이걸 한다).

데이터 누수(leakage)를 조심하라 테스트의 정보가 학습에 새어 들면 성능이 *거짓*으로 좋아진다.
 예: 스케일러를 **전체** 데이터로 적합하면 테스트의 평균·분산이 학습에 스민다. 올바른 순서는 **분할
 먼저, 그다음 학습 데이터로만 전처리 적합**이다. 그래서 이 책의 파이프라인은 `SplitData`를 모델·
 스케일링보다 앞(또는 적절한 위치)에 둔다.

비지도학습: 혼돈 속에서 숲을 찾기

비지도학습은 훨씬 어렵다. **정답이 주어지지 않기** 때문이다. 모델의 성패는 전적으로 들어온 데이터 안의
 패턴·구조·관계를 스스로 추론해 내는 능력에 달려 있다. 목표는 “같은 묶음 안의 대상끼리는 서로 닮고,
 다른 묶음과는 다르게” 데이터를 가르는 것이다.

가장 흔한 비지도학습이 **군집 분석(cluster analysis)** 이다. 정답 레이블 없이도 수많은 점을 의미 있는
 그룹으로 묶어 “나무가 아니라 숲”을 보게 해 준다(7장).

요약 지도학습 = 입력+정답으로 훈련 → 새 입력의 결과 예측(분류·회귀). 비지도학습 = 정답 없이
 구조 발견(군집·차원축소). 어느 쪽이든 워크플로는 *데이터* → *모델* → *평가* → *정제* → *배포* → *피
 드백*의 반복이다.

제2부 · ML Auto Flow 시작하기

3장 — 캔버스 둘러보기

가입 절차 대신, 그냥 열기

Azure ML Studio를 쓰려면 클라우드 구독을 만들고, 워크스페이스를 프로비저닝하고, 가격제를 고르는 일련의 과정을 거쳐야 했다. ML Auto Flow에서 그 모든 단계는 "브라우저로 앱 열기" 한 줄이다. 개발 모드라면 `npm run dev` 후 `http://127.0.0.1:3003`에 접속하면 된다. 로그인도, 결제 정보도 없다.

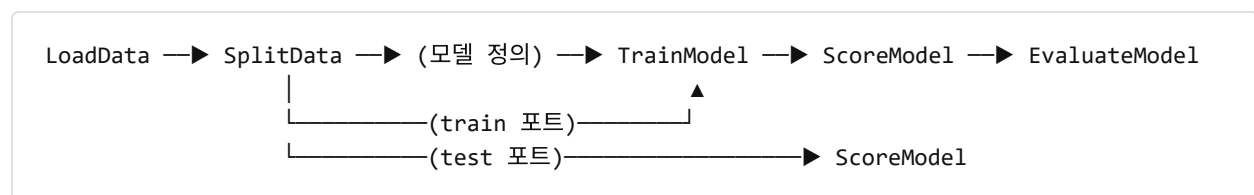
화면의 구성요소

- **모듈 팔레트(툴박스)** — 데이터 적재·전처리·분할·지도/비지도 학습·통계·평가 모듈이 분류별로 놓여 있다.
- **캔버스** — 모듈을 드래그해 배치하고, **포트(입력/출력)**를 선으로 이어 파이프라인을 만든다. 연결선은 곧 *데이터와 모델의 흐름*이다.
- **속성 패널(Properties)** — 선택한 모듈의 파라미터를 편집한다. 각 모듈의 "코드 보기" 탭도 여기에 있다.
- **실행** — 모듈마다 ▶로 개별 실행하거나, 전체를 한 번에 실행한다. 모든 계산은 브라우저 **Pyodide**가 수행한다.
- **미리보기 모달** — 각 결과를 표·차트·요약으로 확인한다. 통계·평가 결과에는 ✨ **AI 해설**이 함께 붙는다.

모듈과 포트, 그리고 흐름

ML Auto Flow의 한 모듈은 워크플로의 한 단계다. 모듈에는 **입력 포트**와 **출력 포트**가 있고, 한 모듈의 출력 포트를 다음 모듈의 입력 포트에 이으면 데이터(또는 학습된 모델)가 그 선을 따라 흐른다.

지도학습 파이프라인의 전형적인 흐름은 이렇다.



`SplitData`는 데이터를 train/test 두 갈래로 나눠 각각 다른 포트에 내보낸다. `TrainModel`은 (모델 정의 + train 데이터)를 받아 **학습된 모델**을 만들고, `ScoreModel`은 (학습된 모델 + test 데이터)로 예측을 채운다. `EvaluateModel`이 그 예측을 정답과 비교해 지표를 낸다. 이 와이어링은 외부 Python으로 내보낼 때도 그대로 보존된다(9장).

파이프라인은 방향 비순환 그래프(DAG)다 모듈은 노드, 연결은 *방향*이 있는 간선이다. 순환(cycle)이 없어야 하며, 앱은 그래프를 **위상정렬**해 “상위 모듈이 끝나야 하위 모듈이 도는” 순서로 실행한다. 한 모듈의 출력은 여러 하위로 갈라질 수 있고 (분기), 학습된 *모델*도 데이터처럼 포트를 따라 흐른다(**모델 포트**). 덕분에 같은 데이터로 여러 모델을 나란히 비교하는 실험을 한 캔버스에 그릴 수 있다. 포트에는 **타입(데이터/모델)**이 있어, 데이터 포트를 모델 입력에 잘못 잇는 실수를 막는다.

 ML Auto Flow 캔버스 — 좌측 모듈 팔레트와, 데이터 로드 → 결측치 처리 → 데이터 스케일링 → 데이터 분할 → 모델 → 학습 → 예측 → 평가로 이어지는 파이프라인

그림 3-1. 실제 ML Auto Flow 캔버스. 각 노드가 한 모듈이고, 노드를 잇는 선이 데이터·모델의 흐름이다. 좌측 상단에 줌·”화면에 맞추기” 컨트롤, 우측 상단에 “전체 실행·”고급기능 실행”이 보인다. (이 화면은 6장 회귀 파이프라인과 동일한 구조다.)

책의 모듈 ↔ 앱의 모듈 (전체 대조는 부록 C)

Azure ML Studio	ML Auto Flow
Dataset 업로드	LoadData
Clean Missing Data	HandleMissingValues
Project Columns	TrainModel 의 feature_columns
Split	SplitData
Train Model	TrainModel
Score Model	ScoreModel
Evaluate Model	EvaluateModel

4장 — 데이터 들여오기와 탐색

LOADDATA — 파일 또는 URL

분석의 출발점은 데이터를 들여오는 일이다. LoadData 는 두 가지 입력을 받는다.

- **로컬 CSV 업로드** — 기기의 파일을 그대로 올린다. 데이터는 브라우저를 떠나지 않는다.
- **URL 직접 로드** — 공개 데이터셋 주소를 입력하면 입력창에서 받아 동일 파서로 처리한다. (책의 Reader 모듈이 웹 URL을 직접 읽던 것과 같은 발상이다. 내부적으로는 fetch로 받아 기존 CSV 파서에 넘기므로, 이후 Pyodide 실행 경로와 재현성에는 전혀 영향이 없다.)

데이터 개요 패널 — 먼저 데이터를 “본다”

좋은 분석가는 모델부터 만들지 않는다. **먼저 데이터를 본다**. ML Auto Flow는 데이터를 올리면 **데이터 개요 패널**이 즉시 다음을 요약한다.

- 행·열의 수,
- 열별 타입(수치형/범주형),
- 열별 결측치 수.

예컨대 성인 소득 데이터를 올리면 `workclass` 같은 열에 결측치가 다수 있음이 강조된다. 책에서 자동차 데이터를 *Visualize*로 살펴 결측치를 발견하고 *Clean Missing Data*로 채웠던 과정과 같다.

결측치 다루기

결측치가 있는 열을 그대로 sklearn 모델에 넣으면 오류가 난다. 두 가지 길이 있다.

1. **결측 없는 수치형 특성만 선택** — 가장 깔끔하게 재현 가능한 길(이 책의 5·6장 예제가 택한 방식).
2. **HandleMissingValues 로 보정** — 평균/중앙값/상수 대체 등으로 결측치를 채운 뒤 사용 (책의 *Clean Missing Data* — Custom Substitution=0 등에 대응).

범주형(직업·학력 등)까지 쓰려면 `EncodeCategorical` 을 체인 앞에 추가해 수치로 바꾼다.

데이터의 종류와 전처리의 논리

열은 측정 수준에 따라 다루는 법이 다르다.

- **수치형(numeric)** — 연속(키·가격) 또는 이산(개수). 사칙연산이 의미를 갖는다.
- **범주형(categorical)** — 명목(직업·색)과 순서(학력·등급). sklearn 모델은 숫자만 받으므로 인코딩이 필요하다.
 - **원-핫 인코딩** — 명목형을 0/1 더미 열로 펼친다(순서가 없으니 정수로 바꾸면 안 된다).
 - **레이블/순서 인코딩** — 순서형을 정수로(작다 < 크다 관계를 보존).

결측치가 왜·어떻게 생겼나 도 처리법을 좌우한다.

- **완전 무작위 결측(MCAR)** — 아무 이유 없이 빠짐 → 행 삭제·평균 대체로 무난.
- **무작위 결측(MAR)** — 다른 관측값에 따라 빠짐 → 조건부 대체(예: KNN)가 더 낫다.
- **비무작위 결측(MNAR)** — 값 자체 때문에 빠짐(고소득자가 소득을 미응답) → 가장 까다롭고, 결측 자체가 신호일 수 있다.

`HandleMissingValues` 는 행 삭제·평균/중앙값/최빈/KNN 대체를 제공한다.

스케일링이 필요한 이유 거리·경사 기반 알고리즘(K-Means·SVM·KNN·신경망)은 *값의 크기*에 민감하다. 연봉(수만)과 나이(수십)를 그대로 두면 연봉이 거리를 지배한다.

`ScalingTransform` (MinMax/Standard/Robust)이 척도를 맞춰 준다 (7장 도매고객 군집의 핵심 교훈). 반대로 **트리 계열은 분할 기준**이라 스케일에 둔감하다 — 그래서 5장 의사결정나무 예제는 스케일링이 결과에 영향을 주지 않는다.

데이터 위생 한 줄 요약 “쓰레기를 넣으면 쓰레기가 나온다(garbage in, garbage out).” 모델을 바꾸기 전에, 데이터를 먼저 보고 고쳐라.

제3부 · 지도학습: 예측 모델 만들기

5장 — 분류: 성인 소득(>50K) 예측

시나리오

이 장에서는 한 사람의 인구통계 정보로 **연 소득이 \$50,000를 넘는지** (>50K) 아닌지 (<=50K)를 예측하는 **이진 분류** 모델을 처음부터 끝까지 만든다. 데이터는 UCI의 *Adult / Census Income* (1994년 미국 인구조사 기반, 32,561행)을 쓴다. 출력이 두 값 중 하나라서 “이진(binary)”이다.

파이프라인

```
LoadData → SplitData(0.8, seed 42) → DecisionTree(max_depth=8)
      → TrainModel → ScoreModel → EvaluateModel(classification)
```

- **특성(결측 없는 수치형 6개):** age, fnlwgt, education-num, capital-gain, capital-loss, hours-per-week.
- **모델:** DecisionTree(max_depth=8). 책은 부스팅 트리 계열을 1순위로 썼는데, ML Auto Flow에는 이에 대응하는 GradientBoosting 모듈이 있어(아래) 동일 예제를 한 단계 더 충실히 재현할 수 있다.
- **앱 사용:** 샘플 Book_AdultIncome_DecisionTree.json 로드 → 실행 (검증 픽스처 13_book_adult_clf).

 분류 파이프라인 — 데이터 로드 → 결측치 처리 → 스케일링 → 분할 → Decision Tree → 학습 → 예측 → 평가

그림 5-1. 의사결정나무 분류 파이프라인을 캔버스에 올린 모습. 모델 노드(Decision Tree)만 Gradient Boosting으로 바꾸면 책의 부스팅 트리 예제로 곧장 전환된다.

의사결정나무는 어떻게 나누나

의사결정나무는 데이터를 **질문**으로 반복해서 쪼갬다(“capital-gain ≤ 7000?”). 각 분할에서 **가장 순도(purity)를 높이는** 질문을 고른다. 순도는 보통 두 척도로 잰다.

- **지니 불순도(Gini)** — 한 노드에서 무작위로 둘을 뽑아 클래스가 다를 확률. 0이면 완전히 순수하다.
- **엔트로피(entropy)** — 정보이론의 무질서도. 분할 전후 엔트로피 감소량이 **정보이득(information gain)**이다.

나무는 정보이득이 큰 질문을 위에 놓고, 잎(leaf)이 충분히 순수해질 때까지 내려간다. **깊을수록 학습 데이터가 잘 외우지만 과적합한다**(2장의 편향-분산). 그래서 `max_depth` 로 깊이를 제한한다 (이 예제는 8). 동점 분할 처리 등에 무작위가 끼므로 `random_state=42` 로 고정해 재현성을 지킨다.

앙상블 — 약한 학습기를 모아 강하게

나무 하나는 불안정하다(데이터가 조금만 바뀌어도 모양이 출렁인다). 이를 모으는 두 전략이 있다.

- **배깅(bagging) / 랜덤 포레스트** — 데이터를 무작위로 여러 번 뽑아 **독립적인** 나무 여럿을 만들고 **평균·다수결**한다. 주로 **분산**을 줄여 안정시킨다.
- **부스팅(boosting) / 그래디언트 부스팅** — 나무를 **순차적으로** 쌓되, 앞 나무가 틀린 부분(잔차)에 다음 나무가 집중하게 한다. 주로 **편향**을 줄여 정확도를 끌어올린다. 책의 1순위 분류기가 이 계열이고, 앱의 `GradientBoosting` 이 대응한다(아래).

검증된 결과 (TEST N = 6,513)

지표	값
정확도(Accuracy)	0.8322
정밀도 Precision (>50K)	0.8136
재현율 Recall (>50K)	0.3947
F1 (>50K)	0.5315

해석. 수치형 6개 특성만으로 정확도 83%에 이른다. 다만 고소득(`>50K`) **재현율이 0.39로 낮다**. 이는 ① 클래스 불균형(고소득자가 소수)과 ② 직업·학력·결혼상태 같은 **범주형을 아직 쓰지 않았기** 때문이다. 책 수준으로 끌어올리려면 — `EncodeCategorical` 로 14개 특성 전부 사용, 부스팅 트리로 교체, 또는 임계값(threshold) 조정이 효과적이다.

분류 평가 깊이 보기 — 정확도만으로는 부족하다

정확도 하나로 모델을 판단하면 위험하다. 100명 중 90명이 저소득인 데이터에서 “무조건 저소득”이라 찍는 모델도 정확도 90%가 나오기 때문이다. 그래서 분류 평가는 여러 각도를 함께 본다.

- **혼동행렬(Confusion Matrix)** — 참양성/거짓양성/참음성/거짓음성의 4칸. 모든 지표의 출발점.
- **정밀도(Precision)** — “양성이라 한 것 중 진짜 양성” 비율. 거짓 경보를 줄이고 싶을 때 중요.
- **재현율(Recall)** — “진짜 양성 중 잡아낸” 비율. 놓치면 안 되는 문제(질병·부정거래)에서 중요.
- **F1** — 정밀도와 재현율의 조화평균. 둘의 균형을 한 수로.
- **ROC 곡선·AUC** — 임계값을 바꿔 가며 본 참양성률 대 거짓양성률. AUC=1이 완벽, 0.5가 무작위.
- **정밀도-재현율(PR) 곡선·Average Precision** — 불균형 데이터에서 특히 유용.

혼동행렬에서 모든 지표가 나온다 양성을 양성이라 하면 **TP**, 음성을 양성이라 하면 **FP**(거짓 경보), 양성을 음성이라 하면 **FN**(놓침), 음성을 음성이라 하면 **TN**. 여기서:

- 정밀도 = $TP / (TP + FP)$ — “양성이라 외친 것 중 진짜”
- 재현율 = $TP / (TP + FN)$ — “진짜 양성 중 잡아낸 것”
- $F1 = 2 \cdot (\text{정밀도} \cdot \text{재현율}) / (\text{정밀도} + \text{재현율})$ — 둘의 **조화평균**(한쪽만 높아선 안 됨)

정밀도와 재현율은 **임계값을 올리면 한쪽이 오르고 다른 쪽이 내려가는** 상충 관계다. 이 예제의 고소득 재현율 0.39는 “임계값 0.5에서 놓침(FN)이 많다”는 뜻 — 임계값을 낮추거나, 부스팅·재표집(SMOTE)으로 끌어올린다. **ROC-AUC** 는 임계값을 전부 훑어 본 “구분 능력”의 단일 요약치라, 임계값 선택과 무관하게 모델을 비교한다.

ML Auto Flow의 `EvaluateModel` 은 분류에서 이 모두를 계산하고, 평가 미리보기 모달에는 **임계값 슬라이더**가 있어 — 슬라이더를 움직이며 정밀도/재현율/F1이 어떻게 변하는지 즉시 볼 수 있다. (이 슬라이더는 미리 계산된 임계값 표에서 고르는 **앱 전용 탐색 도구**이며, 내보낸 Python 코드나 재현성에는 영향을 주지 않는다.) 책의 평가 화면(ROC/AUC·혼동행렬·임계값 조정)을 그대로 옮겨 온 셈이다.

 Evaluation Results 모달 — Selected Threshold 0.50에서 Accuracy 0.9074·Precision·Recall·F1, 그리고 Performance Metrics 표

그림 5-2. `EvaluateModel` 의 분류 평가 결과 모달(의사결정나무, glass 데이터 실행 예). 상단에 **선택한 임계값**과 그에 따른 Accuracy·Precision·Recall·F1, 아래로 임계값별 Performance Metrics 표가 이어진다(스크롤하면 ROC/PR 곡선).

그래디언트 부스팅 — 책의 1순위 분류기에 대응

`GradientBoosting` 모듈은 약한 트리들을 단계적으로 쌓아 오차를 줄여 가는 `sklearn.ensemble.GradientBoostingClassifier/Regressor` 를 감싼다(`random_state=42` 로 결정적). 의사결정나무 자리에 이 모듈을 끼우면 책의 부스팅 트리 예제를 같은 데이터로 재현할 수 있다. 주요 파라미터는 `n_estimators` (트리 수)·`learning_rate` (학습률)·`max_depth` (트리 깊이)이며, 픽스처 `14_gradient_boosting` 으로 외부 재현이 검증되어 있다.

내보낸 동등 PYTHON (검증됨, 핵심 발췌)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

df = pd.read_csv("adult.csv")
features = ["age", "fnlwgt", "education-num", "capital-gain", "capital-loss", "hours-per-week"]
X, y = df[features], df["income"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, train_size=0.8, random_state=42,
                                          shuffle=True)
model = DecisionTreeClassifier(max_depth=8, random_state=42).fit(X_tr, y_tr)
pred = model.predict(X_te)
print("Accuracy =", accuracy_score(y_te, pred))
```

6장 — 회귀: 자동차 가격 예측

시나리오

이번에는 출력이 범주가 아니라 **연속적인 수치** — 자동차의 **가격(price)** — 인 **회귀** 문제다. 데이터는 UCI *Automobile*(1987년 수입차 제원·가격, 201행)을 쓴다. 차폭·엔진 크기·연비 같은 제원으로 가격을 예측하는, 옛날 영업사원이 머릿속으로 하던 일을 모델로 옮긴다.

회귀의 세 갈래(개념)

- **단순 선형회귀** — 하나의 입력으로 하나의 출력을 예측.
- **다중 선형회귀** — 여러 입력으로 하나의 출력을 예측(이 장의 예제가 여기 해당).
- **다변량 선형회귀** — 서로 연관된 여러 출력을 동시에 예측.

선형회귀는 입력 X와 출력 Y의 관계를 직선(또는 초평면)으로 모델링하며, 보통 최소제곱법으로 적합한다. 해석이 쉽고 빠르며 기준선(baseline)으로 훌륭하다.

최소제곱과 R²의 의미 선형회귀는 모든 점에서 예측선까지의 **세로 거리(잔차)**의 **제곱합을 최소화** 만드는 직선을 고른다(최소제곱). 제곱을 쓰는 이유는 부호를 없애고 큰 오차에 더 큰 벌점을 주기 위해서다. **R²(결정계수)**는 “그 직선이 **평균선** 대비 분산을 얼마나 줄였나”이다 — $R^2 = 1 - (\text{잔차제곱합 SS}_{\text{res}} / \text{총제곱합 SS}_{\text{tot}})$. 1에 가까울수록 좋고, 0이면 그냥 평균으로 찍는 것과 다를 바 없다. 이 예제의 R²=0.7561은 가격 변동의 약 76%를 10개 특성이 설명한다는 뜻이다.

파이프라인

```
LoadData → SplitData(0.75, seed 42) → LinearRegression
      → TrainModel → ScoreModel → EvaluateModel(regression)
```

- **특성(결측 없는 수치형 10개):** symboling, wheel-base, length, width, height, curb-weight, engine-size, compression-ratio, city-mpg, highway-mpg. (책은 *Project Columns*로 bore · stroke 를 제외했는데, 본 재현도 두 열을 쓰지 않는다.)
- **앱 사용:** 샘플 Book_Automobile_LinearRegression.json 로드 → 실행 (픽처 11_book_automobile_linreg).

검증된 결과 (TEST N = 51)

지표	값	의미
결정계수 R^2	0.7561	가격 분산의 약 76%를 설명
RMSE	5,129.9	제곱오차 기반 평균 오차(큰 오차에 민감)
MAE	3,487.7	절대오차 평균(이상치에 덜 민감)

해석. 수치형 10개만으로 가격 분산의 76%를 설명한다. 엔진 크기·차폭·공차중량이 가격과 강하게 연동된다. 책처럼 범주형(make, body-style 등)을 인코딩해 더하면 R^2 를 더 끌어올릴 여지가 있다.

회귀 평가 깊이 보기 — 상대오차의 추가

EvaluateModel 은 회귀에서 RMSE·MAE에 더해 ****상대제곱오차(RSE)****와 **상대절대오차(RAE)**를 함께 낸다. 이는 “단순히 평균으로 찍는 모델 대비 얼마나 나은가”를 0~1 척도로 보여 주는 지표로, 책이 제시한 회귀 5종 지표와 정합한다. 척도가 다른 데이터셋들끼리 모델 성능을 비교할 때 특히 유용하다.

RMSE 대 MAE — 어느 오차를 볼까 둘 다 “평균적으로 얼마나 빗나갔나”지만 성격이 다르다.

RMSE(제곱오차 기반)는 큰 오차를 제공해 더 무겁게 벌하므로 *이상치에 민감*하다. **MAE**(절대오차)는 모든 오차를 동등하게 보아 *이상치에 강건*하다. 이 예제에서 $RMSE(5,129.9) > MAE(3,487.7)$ 인 것은, 대부분 잘 맞지만 *몇몇 큰 오차*가 RMSE를 끌어올렸음을 시사한다.

선형회귀가 기대는 가정, 그리고 규제

선형회귀의 계수를 해석하려면 몇 가지 가정이 필요하다 — 관계의 **선형성**, 잔차의 **독립성·등분산성·정규성**. 이를 점검하는 도구가 8장의 통계 모듈(정규성 검정·잔차 분석·VIF)이다. 특성끼리 강하게 상관되면 (**다중공선성**) 계수가 불안정해지는데, **VIF**로 진단하고 상관 높은 열을 줄여 대응한다.

특성이 많거나 과적합이 우려되면 **규제(regularization)**를 더한다. 손실에 계수 크기 벌점을 붙이는 변형으로, 애플의 선형 모델은 다음을 지원한다 — **Ridge(L2)**는 계수를 고르게 줄이고, **Lasso(L1)**는 일부 계수를 0으로 만들어 **변수 선택** 효과를 내며, **ElasticNet**은 둘을 섞는다. 모두 `random_state=42`로 결정적이다.

내보낸 동등 PYTHON (검증됨)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

df = pd.read_csv("imports-85-hdrs.csv")
features = ["symboling", "wheel-base", "length", "width", "height",
            "curb-weight", "engine-size", "compression-ratio", "city-mpg", "highway-mpg"]
X, y = df[features], df["price"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, train_size=0.75, random_state=42,
                                          shuffle=True)
model = LinearRegression().fit(X_tr, y_tr)
pred = model.predict(X_te)
print("R2 =", r2_score(y_te, pred))
print("RMSE =", mean_squared_error(y_te, pred) ** 0.5)
```

회귀 알고리즘 패밀리(개념 지도)

선형회귀는 회귀의 출발점일 뿐이다. 상황에 따라 다음을 고려한다.

- **부스팅 트리 회귀(Gradient Boosting)** — 약한 트리를 단계적으로 쌓아 비선형 관계를 강하게 잡는다(애플 `GradientBoosting`).
- **결정 포레스트(Random Forest)** — 여러 트리의 평균으로 분산을 줄이고 잡음에 강하다.
- **신경망 회귀(Neural Network)** — 전통 회귀가 잡지 못하는 복잡한 비선형을 근사.
- **포아송 회귀(Poisson)** — *개수(count)*를 예측할 때(콜수·청구 건수 등). statsmodels로 제공(8장).
- **분위/순서형 회귀** — 평균이 아닌 분포의 특정 분위, 혹은 순서가 있는 범주를 다룰 때.

제4부 · 비지도학습

7장 — 군집: 도매고객 세분화

정답이 없을 때

분류·회귀가 “정답을 보고 배우는” 지도학습이었다면, 군집은 **정답 없이** 데이터 스스로 자연스러운 묶음을 찾게 하는 비지도학습이다. 목표는 **같은 군집 안끼리는 닮고, 다른 군집과는 다르게** 데이터를 가르는 것. 고객 세분화·이상 탐지·문서 묶기 같은 곳에서 빛을 발한다.

K-MEANS의 원리

가장 널리 쓰이는 군집 알고리즘이 **K-Means** 다. 동작은 직관적이다.

1. 군집 수 K를 정하고, K개의 중심점(centroid)을 임의로 둔다.
2. 각 데이터 점을 **가장 가까운** 중심점에 배정한다.
3. 각 군집의 평균으로 중심점을 옮긴다.
4. 중심점이 더 이상 움직이지 않을 때까지(또는 정해진 반복 수까지) 2~3을 되풀이한다.

”가까움”은 보통 유클리드 거리로 잰다. 거리에 기반하므로, **금액 단위가 큰 열이 결과를 지배하기 쉽다** — 이 점이 아래 해석의 핵심이 된다.

K-Means가 정말 최소화하는 것 K-Means는 **군집 내 제곱합(inertia, WCSS)** — 각 점에서 자기 중심점까지 거리 제곱의 총합 — 을 줄인다. 위 1~4 반복(로이드 알고리즘)은 이 값을 매 단계 감소시키므로 반드시 **수렴** 한다. 단, 초기 중심점에 따라 **지역 최솟값**에 빠질 수 있어, `n_init`로 여러 초기값을 시도해 가장 좋은 것을 고른다(시드 고정으로 결정적). inertia는 K가 커질수록 단조 감소하므로, “**팔꿈치(elbow)**”가 꺾이는 K나 **실루엣 점수** (군집 내 응집도 대 군집 간 분리도)로 적절한 군집 수를 고른다.

시나리오와 파이프라인

UCI *Wholesale customers*(440행, 8열: Channel, Region, Fresh, Milk, Grocery, Frozen, Detergents_Paper, Delicassen)로 도매 거래처를 군집화한다.

```
LoadData → KMeans(n_clusters=4, seed 42) + TrainClusteringModel → ClusteringData
```

이는 책의 *K-Means Clustering + Train Clustering Model + Assign to Clusters* 3모듈 구성과 1:1로 대응한다. **앱 사용:** 샘플 `Book_Wholesale_KMeans.json` (픽스처 `12_book_wholesale_kmeans`).

군집 파이프라인 — Load Data → Select Data → Split Data / Statistics → K-Means → Missing/Scaling Transform → Train Clustering → Clustering Data

그림 7-1. 캔버스에 올린 K-Means 군집 파이프라인. K-Means 노드는 `TrainClusteringModel` 과 함께 학습되고, `ClusteringData`가 각 행에 군집 번호를 배정한다. (앱 내장 “K Means” 샘플과 동일 구조.)

검증된 결과

군집	고객 수
0	277
1	95
2	58
3	10

- 군집 내 제곱합(inertia) $\approx 6.49e+10$.
- **해석.** 다수(277명)는 일반적 소비 패턴, 소수 군집(10명)은 특정 카테고리에 고액을 지출하는 대형 거래처로 보인다. 다만 inertia가 매우 큰 데서 보듯, Fresh·Grocery처럼 금액 단위가 큰 열이 거리 계산을 지배하고 있다. `StandardScaler` (표준화)를 앞에 두면 단위 영향을 줄여 더 균형 잡힌 세분화를 얻을 수 있다. 군집에 정답이 없는 만큼, 해석은 분석가의 몫이다.

거리의 함정 두 가지 ① **척도** — 단위 큰 열이 거리를 지배한다(이 예제 inertia가 $6.49e+10$ 인 까닭). `StandardScaler` 선행이 해법. ② **차원의 저주(curse of dimensionality)** — 차원이 높아질수록 점들 사이 거리가 다 비슷해져 “가까움”이 무의미해진다. 이때 PCA로 핵심 축 2~3개만 남겨 거리·시각화를 회복한다. PCA는 데이터 분산이 가장 큰 방향(주성분 = 공분산행렬의 고유벡터)을 차례로 찾아, 정보 손실을 최소화하며 차원을 줄인다.

군집 패밀리 — K-MEANS를 넘어

ML Auto Flow는 K-Means 외에도 군집 패밀리를 전체코드 내보내기까지 지원한다.

- **DBSCAN** — 밀도 기반. 군집 수를 미리 정하지 않아도 되고, 어디에도 속하지 않는 점은 **노이즈(-1)**로 표시한다. `.predict`가 없는 transductive 모델이라, `ClusteringData`가 `.labels_` 분기로 군집을 배정한다(`eps·min_samples`).
- **계층적(Agglomerative)** — 가까운 점부터 차례로 병합해 트리를 만든다(`n_clusters·linkage`). 역시 transductive.
- **PCA(주성분분석)** — 엄밀히는 차원축소지만, 고차원 데이터를 2~3개 주성분으로 압축해 **군집을 눈으로 보게** 해 준다.

DBSCAN·계층적은 결정적이라 시드가 필요 없고, K-Means·PCA는 `random_state=42`로 고정된다.

군집 고르기 가이드 구형(동근) 군집·군집 수를 안다 → **K-Means**. 모양이 불규칙하거나 노이즈가 있다 → **DBSCAN**. 군집 수를 모르고 계층 구조가 궁금하다 → **계층적**. 시각화·전처리로 차원을 줄이고 싶다 → **PCA**.

제5부 · 책 너머로: 통계와 진단

8장 — 통계 분석과 가설검정

Azure ML 책은 예측 모델에 집중하느라 **고전 통계**를 거의 다루지 않는다. ML Auto Flow는 바로 이 지점을 확장한다. 모델을 만들기 **전에** 데이터를 진단하고, 가정을 검증하고, 관계를 해석하는 통계 도구를 모듈로 제공한다.

- **기술통계(Descriptive)** — 평균·분산·분위수·왜도/첨도로 데이터의 윤곽을 잡는다.
- **상관분석(Correlation)** — 변수 쌍의 선형 관계 강도. 다중공선성의 첫 신호.
- **정규성 검정(Normality)** — Shapiro-Wilk 등으로 “정규분포 가정”이 타당한지 점검.
- **가설검정(Hypothesis Testing)** — t-검정·카이제곱 등으로 집단 차이·독립성을 통계적으로 판단.
- **이상치 탐지(Outlier)** — IQR·z-score로 튀는 값을 찾아낸다.
- **VIF(분산팽창계수)** — 회귀 특성 간 다중공선성을 정량화. VIF가 크면 계수 해석이 불안정.
- **statsmodels 회귀** — **OLS**(상세한 계수·p값·신뢰구간), **로지스틱**, **포아송**(개수 회귀). sklearn이 “예측”에 강하다면, statsmodels는 “**설명과 추론**”에 강하다.

각 결과 모달은 표·차트와 함께 **🔥 AI 한국어 해설**을 제공해, p값이나 VIF 같은 수치를 “그래서 무슨 뜻인가”로 풀어 준다. 통계 모듈군 역시 외부 Python으로 재현 가능하다 (예: OLS 픽스처

```
05_statsmodels_ols ).
```

가설검정의 문법 — p값이 뜻하는 것 가설검정은 “**귀무가설 H_0** (차이·관계가 없다)”를 기본값으로 두고, 데이터가 그 가정 아래서 얼마나 **희귀**한지를 본다. **p값**은 “ H_0 가 참일 때, 관측된 만큼(또는 그 이상)의 차이가 우연히 나올 확률”이다. $p < \text{유의수준 } \alpha$ (보통 0.05)면 H_0 를 기각한다. 흔한 오해 두 가지 — p값은 **효과의 크기**가 아니며, **상관은 인과가 아니다**. 또한 H_0 가 참인데 기각하면 **1종 오류**(거짓 양성), 거짓인데 못 기각하면 **2종 오류**(놓침)다. sklearn이 **예측**에 강하다면, statsmodels(OLS/로지스틱/포아송)는 계수·p값·신뢰구간으로 **설명·추론**에 강하다.

 **Statistics Preview** 모달 — iris 데이터의 열별 기술통계(Count·Mean·Std·Median·Min·Max)를 표로 보여준다

그림 8-1. `Statistics` 모듈을 실행한 결과 미리보기 모달(iris 데이터). 열별 비결측 수·타입·평균·표준편차·중앙값·최소/최대를 표로 제시하며, 우측 상단 **🔥** 버튼으로 AI 해설을 받을 수 있다(아래로 스크롤하면 상관관계 히트맵).

왜 통계가 먼저인가 회귀 계수를 신뢰하려면 다중공선성(VIF)을 봐야 하고, t-검정을 쓰려면 정규성을 점검해야 한다. 통계는 모델의 **전제**를 지키게 해 주는 안전장치다.

제6부 · ML Auto Flow의 차별화된 강점

9장 — Python 코드 내보내기과 재현성 ★

이 장이 ML Auto Flow의 **심장**이다. 캔버스에서 만든 파이프라인은 클릭 한 번으로 **외부에서 그대로 실행되는 standalone Python**으로 내보내진다. 앱 안에서만 도는 블랙박스가 아니라, 여러분이 들고 나가 Jupyter·VS Code·서버 어디서든 돌릴 수 있는 **진짜 코드**가 된다.

 전체 파이프라인 코드 패널 — 내보낸 Python의 머리말에

그림 9-1. “전체 파이프라인 코드” 패널. 캔버스 파이프라인이 단계별 주석이 붙은 실행 가능한 Python으로 내보내진다. 머리말이 “외부 Python에서 그대로 실행 가능 · 동일 결과 재현”임을 명시한다.

재현성 불변식

- 모든 무작위 단계에 `random_state=42` 고정 — 데이터 분할, 트리 초기화, K-Means 중심점 등. 따라서 같은 데이터·같은 코드면 외부 Python에서도 **같은 결과**가 나온다.
- 객체 파라미터는 Python 리터럴로 — JS의 `true`가 아니라 Python의 `True`로 직렬화된다.
- 포트 와이어링 보존 — `SplitData`의 train/test 포트, 모델 생성 → Train → Score의 변수 매핑이 내보낸 코드에 그대로 반영된다.

무엇이 “같은 결과”를 깨뜨리나 — 비결정성의 원천 같은 코드가 다른 답을 내는 흔한 이유는 셋이다. ① 무작위성 — 분할·초기화·표본추출의 난수. → 모든 단계에 시드 고정으로 차단. ② 순서 의존 — 딕셔너리·집합·그룹 순서가 입력에 따라 흔들림. → 사용자/아이템·컬럼을 정렬하고 안정 정렬을 써서 차단(12장 추천이 그 예). ③ 부동소수점·라이브러리 버전 — 합산 순서나 BLAS 구현 차이. → 결정적 알고리즘·고정 연산 순서로 최소화. 이 책의 모든 예제는 ①②를 설계로 제거했고, ③은 같은 환경에서 byte-identical로 검증된다.

자동 회귀 검증 — 약속을 기계가 지킨다

“재현된다”는 말을 사람의 선의에 맡기지 않는다. 다음 한 줄이 그것을 강제한다.

```
npm run verify:pipelines
```

이 명령은 각 픽스처의 내보낸 코드를 외부 Python으로 **2회 실행**해 출력이 ****바이트 단위로 동일(byte-identical)****한지 단언한다. 현재 **15/15 PASS**이며, 다음을 모두 포함한다 — 회귀·분류·전처리·statsmodels(OLS)·신경망·군집 4종(K-Means/DBSCAN/계층적/PCA)·책 예제 3종(자동차/도매고객/성인소득)·그래디언트 부스팅·하이퍼파라미터 스왑·추천(협업 필터링). 검증 픽스처 목록과 데이터는 부록 B에 있다.

하이퍼파라미터 스위프 — 자동 튜닝도 결정적으로

`SweepParameters` 모듈은 `GridSearchCV` (정수 `cv` → 완전 결정적)로 최적 추정기를 찾아 `Train/Score/Evaluate`에 그대로 연결한다. 손으로 파라미터를 바꿔 가며 ▶를 누르는 대신, 격자 탐색을 한 번에 돌리되 **재현성은 그대로** 지킨다(픽스처 `15_sweep_gridsearch`).

재현성이 왜 중요한가 분석 결과를 동료·감독기관·미래의 나에게 “이대로 돌리면 같은 숫자가 나온다”고 보일 수 있어야 비로소 신뢰가 생긴다. ML Auto Flow는 그 신뢰를 *기본값*으로 만든다.

10장 — AI 보조 기능: 코드·결과·오류를 한국어로 ★

ML Auto Flow는 사용자별 **로컬 API 키**(브라우저 `localStorage`에 저장, 번들에 하드코딩하지 않음)로 동작하는 AI 헬퍼를 제공한다. 데이터 위에서 일하다 막히는 지점마다 그 자리에서 설명을 받을 수 있다.

- **코드 해설** — 내보낸 Python이 무슨 일을 하는지 한국어로 풀어 준다.
- **결과 해석** — 통계·평가 결과 모달에서 ✨ 버튼으로 “이 R^2/p 값/AUC가 의미하는 바”를 설명.
- **오류 진단** — 모듈 실행이 실패하면 원인과 수정 방법을 제안.
- **AI 파이프라인 생성** — 분석 목표나 데이터로부터 파이프라인 초안을 만들어 캔버스에 올려 준다.

고급기능 비밀번호 게이트 AI·PPT·코드 보기/내보내기·API 키 설정 같은 *API·코드 관련 고급기능*은 비밀번호로 잠겨 있다. 일반 사용자는 모듈 배치·연결·실행·결과 미리보기까지 자유롭게 쓰고, 고급기능은 해제한 사용자만 쓴다. 잠긴 버튼은 🔒 배지와 흐림으로 구분된다. (수업·전시 환경에서 핵심 기능은 열쇠 비용·노출은 통제하는 장치.)

11장 — 모델을 서비스로: 스코어링 코드 내보내기

책의 또 다른 큰 주제는 “모델을 **웹 서비스**로 만들어 어디서나 호출하기”였다. Azure에서는 스코어링 실험을 만든 뒤 웹 서비스로 게시(Publish)하면 REST/JSON 엔드포인트가 만들어졌다.

ML Auto Flow는 같은 목표를 **클라우드 없이** 이룬다. 학습된 모델 파이프라인에서 **스코어링 배포 코드**를 내보내면 다음이 한 번에 생성된다.

- 학습 모델을 `joblib` 로 **저장/로드**하는 스니펫,
- **FastAPI / Flask** 스코어링 **엔드포인트** 골격(요청 → 예측 → 응답),
- 요청/응답 **JSON 샘플**.

여러분은 이 코드를 그대로 서버에 올려 자신의 인프라에서 모델을 서비스할 수 있다. Azure가 클라우드 종속으로 풀던 “배포” 단계를, ML Auto Flow는 *이식 가능한 코드*로 푼다. (스코어링/배포 내보내기는 고급기능 게이트로 보호된다.)

12장 — 추천 시스템: 협업 필터링과 행렬분해

추천 엔진은 왜 강력한가

”이 상품을 산 사람은 저 상품도 샀습니다.” 추천은 오늘날 가장 널리 쓰이는 예측 분석이다. 전자상거래 매출의 두 자릿수 향상, 스트리밍 시청의 큰 뛴이 추천에서 나온다고 알려져 있다. 사람은 본능적으로 “남들이 무엇을 하는가”에 끌리고, 추천은 그 심리를 데이터로 포착한다.

두 가지 방법론

- **협업 필터링(Collaborative Filtering)** — *나와 비슷한 사용자들이 좋아한 것을* 추천한다. 사용자×아이템 평점 행렬의 패턴에서 추천을 끌어낸다(아마존의 “Also Bought”).
- **콘텐츠 기반(Content-based)** — *내가 좋아한 아이템과 닮은 것을* 추천한다(같은 장르의 영화 등).

실무에서는 둘을 섞고, 군집 같은 다른 알고리즘과도 결합한다. 책이 다른 추천기는 사용자/아이템의 메타데이터까지 활용하는 베이지안 모델이었다.

협업 필터링은 다시 둘로 나뉜다.

- **메모리 기반(이웃법)** — 평점이 비슷한 사용자(또는 아이템)를 *직접* 찾아 그들의 평점을 빌려온다. 단순하지만, 사용자·아이템이 많아지면 모든 쌍의 유사도 계산이 무거워지고 희소성에 약하다.
- **모델 기반** — 평점 행렬에서 숨은 구조를 학습한 **모델**로 빈칸을 채운다. 대표가 아래의 **행렬분해**다. ML Auto Flow의 `Recommender`가 이 길을 택한다.

희소 행렬과 콜드 스타트

추천의 본질은 **희소(sparse) 행렬** 문제다 — 사용자는 많고 아이템도 많은데, 실제로 평가된 칸은 극히 일부다. 비어 있는 칸의 평점을 메우는 것이 곧 추천이다. 새 사용자/새 아이템은 데이터가 거의 없어 추천이 어려운데, 이를 **콜드 스타트** 문제라 한다.

행렬분해 — 잠재요인으로 빈칸 메우기

추천의 핵심 아이디어는 의외로 단순하다. 거대하고 듬성듬성한 평점 행렬 **R**(사용자 × 아이템)을 **두 개의 작은 행렬의 곱**으로 근사하는 것이다.

$$R \approx W \cdot H$$

(사용자×아이템) (사용자×k) · (k×아이템)

여기서 **k개의 잠재요인(latent factor)** 이 핵심이다. 영화라면 “액션성·로맨스 함량” 같은, *데이터가 스스로 찾아낸* 숨은 취향 축이다. **W**의 한 행은 “이 사용자가 각 취향 축을 얼마나 좋아하나”, **H**의 한 열은 “이 아이템이 각 취향 축을 얼마나 담았나”를 나타낸다. 둘을 곱하면 — 즉 사용자의 취향과 아이템의 성격을 내적하면 — **비어 있던 칸의 예상 평점**이 복원된다. 수천 개 아이템을 단 k개 축으로 압축하므로(보통 k는 수십), 희소성과 차원의 저주를 동시에 누른다.

왜 NMF(비음수)인가 행렬분해에는 SVD·ALS 등 여러 방식이 있다. **NMF(Non-negative Matrix Factorization)** 는 **W·H의 모든 원소를 0 이상으로 강제한다**. 평점·구매량·이용횟수처럼 **음수가 없는 데이터에선** 이 제약이 자연스럽고, 분해 결과가 “각 취향을 얼마나 가졌나”라는 **더하기식(부분의 합)** 으로 해석되어 직관적이다. ML Auto Flow가 NMF를 고른 또 하나의 이유는 **Pyodide 호환** — 전용 추천 패키지 `surprise` 는 브라우저 Python에 없지만, NMF는 scikit-learn에 들어 있어 *브라우저에서도, 외부 Python에서도* 똑같이 돈다.

ML AUTO FLOW의 `RECOMMENDER` — NMF 행렬분해

ML Auto Flow의 `Recommender` 모듈은 **자기완결적(self-contained) 협업 필터링** 추천기다. `(user, item, rating)` 삼중쌍에서 사용자×아이템 행렬을 만들고, 이를 **NMF(비음수 행렬분해)** 로 분해한다.

- `init='nndsvda' + random_state=42` 고정 → **완전 결정적**(같은 입력이면 같은 추천).
- NMF는 결측을 0으로 채운 비음수 행렬을 잠재요인 `n_components` 개로 분해해, 빈 칸의 평점을 복원한다.
- 사용자별 상위 `top_n` 아이템을 추천으로 돌려준다.
- **Pyodide 호환을 위한 설계 결정**: 외부 패키지 `surprise` 는 Pyodide에 없으므로, Pyodide의 scikit-learn에 들어 있는 **NMF/TruncatedSVD**만으로 추천을 구현했다. 덕분에 브라우저 안에서도, 외부 Python에서도 똑같이 돈다(픽스처 `16_recommender`, 데이터 `ratings_small.csv` — `user_id, item_id, rating`).

이로써 책의 추천 장(레스토랑 평점 추천)의 학습 목표 — *희소 평점 행렬에서 추천을 생성하기* — 를 클라우드 추천 서비스 없이 재현한다. 보험/헬스케어 맥락에서는 상품 교차판매 추천 등으로 확장할 여지가 있다.

 Recommender 결과 — 사용자별 Top-N 추천(`user_id·rank·item_id·predicted_rating`)

그림 12-1. `Recommender` (NMF) 실행 결과 미리보기. `(user_id, item_id, rating)` 평점 테이블에서 사용자별 상위 추천 아이템과 예측 평점을 돌려준다(앱 샘플 `Book_RestaurantRatings_Recommender`, 데이터 `ratings_small.csv`). 같은 입력이면 같은 추천(결정적, `random_state=42`).

예제: 레스토랑 평점 추천 — 처음부터 끝까지

책의 추천 장은 손님(사용자)이 식당(아이템)에 매긴 별점에서 *아직 안 가 본 식당*을 추천한다. ML Auto Flow는 이를 신규 샘플 `Book_RestaurantRatings_Recommender` 로 재현한다. 두 모듈뿐인 가장 단순한 파이프라인이지만, 그 안에서 행렬분해가 어떻게 도는지 한 단계씩 따라가 보자.

```
LoadData(ratings_small.csv) —▶ Recommender(user_col, item_col, rating_col,
n_components=2, top_n=3)
```

1단계 — 데이터: 긴(long) 형식의 평점

`ratings_small.csv` 는 한 행이 “한 사용자가 한 아이템에 준 평점” 하나인 **긴 형식**이다(5명 × 6아이템, 25개 평점).

```
user_id,item_id,rating
u1,i1,5
u1,i2,3
...
u5,i6,4
```

이걸 **사용자 × 아이템** 표로 펼치면(피벗) 아래처럼 군데군데 빈칸(아직 평가 안 함)이 생긴다 — 바로 **희소 행렬**이다. 추천의 목표는 이 빈칸을 채워 *높게 예측된 미평가 아이템*을 골라 주는 것이다.

	i1	i2	i3	i4	i5	i6
u1	5	3	4	1	2	.
u2	4	5	2	2	1	.
u3	3	2	5	.	4	3
u4	5	4	2	3	.	4
u5	5	.	3	4	5	4

2단계 — 행렬 만들기(결정성의 첫 단추)

`Recommender` 는 먼저 같은 `(user, item)` 중복을 평균으로 합치고, 피벗한 뒤 **행·열을 정렬**한다. 입력 CSV의 행 순서가 어떻게 늘 같은 **행렬**이 되게 하기 위해서다(재현성의 토대). 빈칸은 NMF가 음수·NaN을 못 받으므로 **0으로 채워**되, “원래 빈칸이었다”는 위치(`rated_mask`)는 따로 기억해 둔다 — 추천할 때 *이미 본 아이템*을 제외하는 데 쓴다.

3단계 — NMF로 분해하고 빈칸 복원

표준화된 행렬 R 을 **NMF**로 $W \cdot H$ 로 분해한다($n_components=2 \rightarrow$ 잠재요인 2개). 재구성 $\hat{R} = W \cdot H$ 가 모든 칸의 예측 평점이다. 핵심은 결정성을 보장하는 두 선택이다.

```
from sklearn.decomposition import NMF
# init='nndsvda'(결정적 초기화) + random_state=42 => 같은 입력이면 항상 같은 W·H
model = NMF(n_components=2, init='nndsvda', random_state=42, max_iter=500)
W = model.fit_transform(R_filled)      # 사용자×2
H = model.components_                  # 2×아이템
R_hat = W @ H                          # 재구성(예측) 평점
```

왜 `init='nndsvda'`인가 — 근거 NMF는 보통 무작위 초기값에서 시작해 반복 최적화하므로, 그냥 두면 실행마다 결과가 달라진다. `nndsvda`는 데이터의 SVD에서 **결정적으로** 초기값을 잡는 방법이라, `random_state=42`와 함께 쓰면 분해가 **완전히 재현 가능**해진다. 이 책의 재현성 불변식(9장)을 추천에서도 지키는 장치다.

4단계 — 사용자별 Top-N 고르기

각 사용자 행에서 예측 평점이 높은 순으로 줄을 세우되, **이미 평가한 아이템은 제외**(`-inf` 처리)하고 **아직 안 본 상위 `top_n` 개**(여기선 3개... 단, 미평가 칸이 적은 사용자는 그만큼만)를 추천한다. 동점일 때는 아이템 위치로 **안정 정렬**해, 버전이 바뀌어도 순위가 흔들리지 않게 한다.

5단계 — 결과 읽기

그림 12-1의 결과(각 사용자 1순위 발췌):

user_id	rank	item_id	predicted_rating
u1	1	i6	1.7344
u2	1	i6	0.7492
u3	1	i4	2.0688
u4	1	i5	1.4336
u5	1	i2	0.3942

해석. 예컨대 u3는 i1·i2·i3·i5·i6을 이미 평가했고 유일한 빈칸이 i4였는데, 모델은 그 i4를 2.07로 가장 높게 예측해 1순위로 추천했다. u1·u2는 공통적으로 미평가였던 i6가 1순위로 떠올랐다 — 취향이 닮은 다른 사용자들의 i6 평점 패턴이 잠재요인으로 흘러든 결과다. **예측 평점의 절대값은 원래 별점 척도 (1~5)와 정확히 일치하지 않는다**(NMF 재구성은 0 채움·요인 수에 따라 압축되므로). 중요한 건 **값 자체**가 아니라 **사용자 안에서의 상대 순위** — “이 사용자에게 무엇을 먼저 권할까”다.

이 예제의 파라미터를 그렇게 정한 근거

- `n_components=2` — 사용자·아이템이 각 5·6개로 작아, 잠재요인은 행렬의 작은 쪽(5) 미만이어야 한다. 2개면 “두 갈래 취향”을 잡기에 충분하고 과적합도 피한다(실데이터에선 수십으로 키운다).
- `top_n=3` — 한 화면에 부담 없이 보여 줄 추천 수. 미평가 칸이 3개 미만인 사용자는 가능한 만큼만 나온다.
- 시드·초기화 고정 — **재현성**. 같은 평점이면 누가 언제 돌려도 같은 추천이 나와야 신뢰가 생긴다.

책과의 대응, 그리고 보험·헬스케어로의 확장

책(Matchbox Recommender)	ML Auto Flow(<code>Recommender</code>)
사용자×아이템 평점 삼중쌍 학습	<code>(user_id, item_id, rating)</code> 긴 형식 입력
베이지안 잠재요인 모델	NMF 잠재요인(<code>n_components</code>)
평점 예측 / 아이템 추천	<code>R̂</code> 예측 → 미평가 Top-N
클라우드 웹 서비스	브라우저 Pyodide + 내보낸 standalone Python

도메인을 바꾸면 그대로 보험·헬스케어 추천이 된다 — `user_id` =가입자, `item_id` =상품/서비스, `rating` =가입·이용 강도로 두면 **상품 교차판매(cross-sell)** 추천이 된다(JMDC 시나리오와 연결).

재현(근거 보강)

- **자동 검증**: 검증 픽스처 `16_recommender` 가 이 파이프라인의 내보낸 코드를 외부 Python으로 2회 실행해 **byte-identical**임을 단언한다(`npm run verify:pipelines` , 15/15 중 하나).
- **앱에서**: “샘플”에서 `Book_RestaurantRatings_Recommender` 로드 → `LoadData` 선택 후 Examples에서 `ratings_small.csv` 더블클릭 로드 → 전체 실행 → 결과 보기.
- **데이터 자산**: 샘플은 Supabase(`app_section=ML` , 두 앱 공유)에, 데이터는 LoadData Examples에 등록되어 있다(부록 B).

추천은 어떻게 평가하나(참고) 분류·회귀와 달리 추천은 “안 본 것”을 맞히는 문제라 평가가 까다롭다. 흔히 알려진 평점 일부를 가려 (held-out) 예측 평점과의 **RMSE**를 보거나, 추천 목록의 적중률 **precision@k / recall@k**, 순위 품질 **NDCG**를 쓴다. 본 예제는 모듈의 동작·**재현성** 시연에 초점을 두므로 정량 평가 단계는 생략했고, 향후 held-out 평가를 `EvaluateModel` 계열로 확장할 수 있다.

13장 — 모델 재학습과 지속학습

피드백 루프가 모델을 살린다

머신러닝 모델은 한 번 만들고 끝이 아니다. 세상이 바뀌면 데이터 분포도 바뀐다(개념 표류, concept drift). 1994년 인구조사로 학습한 소득 모델을 오늘 그대로 쓸 수 없는 것과 같다. 그래서 **새 데이터로 모델을 다시 학습(retrain)** 하는 피드백 루프가 모델의 수명을 결정한다. 책의 마지막 장의 핵심은 — 사람의 개입 없이 *프로그램적으로* 모델을 재학습하는 “지속학습” 워크플로였다.

언제 다시 학습해야 하나 — 표류의 두 얼굴 모델이 늙는 데는 두 가지가 있다. **데이터 표류(data drift)** — 입력 분포가 변함(새 고객층 유입). **개념 표류(concept drift)** — 입력과 정답의 *관계*가 변함(같은 특성인데 결과가 달라짐). 이를 감지하려면 운영 중 **성능 지표를 모니터링**하고(정확도·AUC 하락), 입력 분포를 과거와 비교한다. 재학습 전략도 둘 — 새 데이터를 모아 주기적으로 새로 학습하는 **배치 재학습**, 들어오는 즉시 조금씩 갱신하는 **온라인 학습**. 새 모델을 바로 교체하지 않고 기존 모델과 나란히 비교(**챔피언-챌린저**, A/B)한 뒤 더 나은 때만 승격하는 것이 안전하다.

ML AUTO FLOW의 재학습 — 버전이 매겨진 모델 스냅샷

ML Auto Flow에는 이미 “저장한 파이프라인을 새 데이터로 다시 실행”하는 흐름이 있다. 여기에 가산적으로 더해진 기능이 **버전 스냅샷 번들 내보내기**다. 학습 모델 파이프라인에서 다음을 묶어 낸다.

- **메타데이터 헤더** — 모델 타입 / 피처 컬럼 / 데이터 소스 참조 / (있으면) 지표 / **VERSION 라벨**,
- `joblib` **모델 저장·로드 스니펫**(학습된 `trained_model` 전제),
- 버전 메타를 **JSON 사이드카**로 함께 기록하는 코드.

새 데이터가 도착하면 같은 파이프라인을 `v1 → v2 → v3`로 다시 돌려 버전이 매겨진 스냅샷을 쌓아 간다. 어느 버전이 어떤 데이터·지표로 만들어졌는지 추적할 수 있어, “지속학습”을 손으로 실천하는 길이 된다.

재현성 메모(중요) 버전/타임스탬프 라벨은 **절대 자동 생성(예: 현재 시각)**으로 만들지 않는다. UI 입력 필드에서 받은 값(기본 `v1`)을 그대로 쓴다. 따라서 같은 입력이면 같은 출력(결정적)이며, 이 기능은 기존 모듈 동작·실행 경로·검증을 전혀 건드리지 않고 메타데이터를 *읽기만* 한다.

부록

부록 A — 모듈 레퍼런스

각 모듈의 상세(제목·역할·입출력·사용 시점·연결·흔한 오류·주의)는 앱의 `moduleDescriptions.ts` 에 정의되어 있어 그대로 발췌·갱신할 수 있다. 카테고리별 대표 모듈:

- **데이터 I/O:** `LoadData` (파일·URL), 데이터 개요 패널, `DataPreview`.
- **전처리:** `HandleMissingValues`, 열 선택/필터, `EncodeCategorical`, `ScalingTransform`, `SplitData`.
- **지도학습 모델:** `LinearRegression`, `LogisticRegression`, `DecisionTree`, `RandomForest`, `GradientBoosting`, `SVM`, `KNN`, `NeuralNetwork`.
- **모델 연산:** `TrainModel`, `SweepParameters`, `ScoreModel`, `EvaluateModel` (ROC/AUC·회귀지표).
- **비지도:** `KMeans`, `DBSCAN`, `Hierarchical`, `PCA`, `TrainClusteringModel`, `ClusteringData` (군집 할당).
- **통계·검정:** 기술통계·상관·정규성·가설검정·이상치·VIF·OLS/Logistic/Poisson(statsmodels).
- **추천:** `Recommender` (협업 필터링/NMF).

부록 B — 예제 데이터셋과 검증 픽스처

데이터셋(`verify/datasets/`) — 모두 UCI Machine Learning Repository 등 공개 데이터.

파일	내용	비고
<code>imports-85-hdrs.csv</code>	자동차 제원·가격 (201행)	? → 결측, price 결측 행 제거, 헤더 부여
<code>adult.csv</code>	성인 인구조사 소득 (32,561행)	헤더 부여, 공백 제거, ? → 결측
<code>wholesale_customers.csv</code>	도매고객 연간 지출 (440행, 8 열)	원본 헤더
<code>ratings_small.csv</code>	사용자-아이템 평점	<code>user_id, item_id, rating</code>

검증 픽스처(`verify/pipelines/`) — 15종, `npm run verify:pipelines` **15/15 PASS:**

01 회귀(선형)	02 분류(트리)	03 전처리·분석
05 statsmodels OLS	06 신경망 분류	07 K-Means
08 PCA	09 DBSCAN	10 계층적
11 자동차 회귀(책)	12 도매고객 군집(책)	13 성인소득 분류(책)
14 그래디언트 부스팅	15 하이퍼파라미터 스윙	16 추천(협업 필터링)

(04는 결번. 클러스터링은 *transductive* 분기를 포함해 모두 외부 Python 2회 byte-identical로 검증됨.)

앱 로드용 샘플(4종): Book_Automobile_LinearRegression, Book_AdultIncome_DecisionTree, Book_Wholesale_KMeans, Book_RestaurantRatings_Recommender (신규, 12장) — 앱 "샘플" 패널에서 바로 보이고(Supabase autoflow_samples, app_section=ML, 두 앱 공유), 한 번 클릭으로 캔버스에 재현된다.

LoadData Examples(데이터) 등록: 위 4종이 쓰는 데이터(imports-85-hdrs.csv · adult.csv · wholesale_customers.csv · ratings_small.csv)를 Examples_in_Load/ + public/examples-in-load.json (빌드)에 추가해, LoadData 모듈의 **Examples** 목록에서 더블클릭으로 로드할 수 있다. 따라서 **샘플 로드** → **데이터 더블클릭** → **전체 실행** 만으로 책의 각 예제를 끝까지 돌릴 수 있다.

부록 C — Azure ML Studio ↔ ML Auto Flow 용어 대조표

개념/단계	Azure ML Studio	ML Auto Flow
작업 공간	Workspace(클라우드)	브라우저 탭 (가입·과금 없음)
실험 캔버스	Experiment 디자이너	캔버스
데이터 적재	Dataset 업로드 / Reader(URL)	LoadData (파일·URL)
결측 처리	Clean Missing Data	HandleMissingValues
열 선택	Project Columns	TrainModel 의 feature_columns
분할	Split	SplitData
모델 정의	Initialize Model	LinearRegression / DecisionTree / KMeans 등
지도학습 학습	Train Model	TrainModel
군집 학습	Train Clustering Model	TrainClusteringModel
채점	Score Model	ScoreModel
군집 배정	Assign to Clusters	ClusteringData
평가	Evaluate Model	EvaluateModel
부스팅 트리	Boosted Decision Tree	GradientBoosting
추천	Matchbox Recommender	Recommender (NMF)
배포	Publish Web Service(클라우드)	스코어링 코드 내보내기(joblib+FastAPI/Flask)
재학습	Retraining API(배치)	버전 스냅샷 내보내기
실행 환경	Azure 클라우드	브라우저 내 Python(Pyodide)

부록 D — 변경 이력

기능 연혁은 `CLAUDE.md` (변경 이력 표)와 `HISTORY.md` 에 누적된다. 책자 관점의 주요 이정표:

- 책 예제 3종(자동차 회귀·성인소득 분류·도매고객 군집) 앱 재현 + 검증 픽스처.
- 횡단 공통 I/O 개선 6종(샘플 메타·URL 로더·데이터 개요·스코어링 내보내기 등).
- `EvaluateModel` ROC/AUC·PR 곡선·임계값 슬라이더, 회귀 RSE/RAE.
- `GradientBoosting` (부스팅 트리 대응), `SweepParameters` (GridSearchCV).

- 군집 패밀리 완성(K-Means·DBSCAN·계층적·PCA) 전체코드 내보내기.
- **Recommender** (협업 필터링/NMF) + 재학습 버전 스냅샷 내보내기.
- 책 예제 4종(+추천 신규)을 **Samples 패널(Supabase)** 과 **LoadData Examples(데이터)** 에 등록 — 앱 내 즉시 사용.
- 검증 픽스처 **15/15 PASS** 유지(외부 Python 2회 byte-identical).

제작 메모 (판단 사항)

- **출처/저작권:** 본 책자는 *Microsoft Azure Essentials: Azure Machine Learning*(Jeff Barnes, Microsoft Press, 2015)의 **학습 구조와 개념을 길잡이로 삼되**, 모든 본문은 ML Auto Flow 맥락에서 **새로 서술했다**(원문 텍스트 복제 없음). 예제 데이터는 UCI 등 공개 데이터이며, 모든 수치·코드는 앱의 **verify** 하네스로 검증된 것만 사용했다.
- **단일 진실원천(SSOT):** 이 Markdown이 원본이다. **make-pdf** 로 출판 품질 PDF, **app-doc-ppt** 로 요약 PPT를 파생할 수 있다.
- **스크린샷:** Playwright(MCP)로 **npm run dev** (127.0.0.1:3003)의 실제 화면을 캡처해 3·5·7·8·9장에 삽입했다 (**images/** — 캔버스 파이프라인 3종[선형회귀·의사결정나무·K-Means], 내보낸 코드 패널 1종, Statistics 결과 모달 1종, **EvaluateModel 분류 평가 결과 모달 1종**[임계값·Accuracy/Precision/Recall/F1]). 군집 분포(ClusteringData) 결과 모달은 K-Means 내장 샘플의 **인앱 실행기 결함**(Select Data가 인앱에서 출력 미생성 + 평가 차트의 React 렌더 루프)으로 캡처 보류. 내보낸 코드 경로의 **SelectData** 빈 선택은 전체통과로 수정해 외부 재현은 정상(verify 15/15). 인앱 실행기 버그는 별도 디버깅 대상.
- **두 앱:** 본문 1~13장은 베이스/JMDC 공통이며, JMDC 헬스케어 사례(코호트·발생률·생존분석 등)는 JMDC 책자의 별도 부록으로 분리하는 것을 권장한다.